

MAGNA · SECURITY REVIEW

Magna Merkle Vester.

A security review of Merkle Vester.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Magna	Merkle Vester	Guardian	August 11 – August 13, 2025	August 20, 2025

§ 00

Contents.

34 pages — overview, confidence ranking, scope, 13 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 13 findings	P. 08
§ 05	Disclaimer	P. 33
§ 06	About Guardian	P. 34

§ 01

Overview.

Guardian reviewed Merkle Vester across a single review round. The final report includes 1 Medium, 6 Low, 6 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	magna-eng/protocol
CLIENT	METHODOLOGY
Magna	Smart contract · Manual · Main Review
PROJECT	MAIN REVIEW COMMIT
Merkle Vester	6d1a8217cc71
PREPARED BY	
Guardian	
AUDIT WINDOW	
August 11 – August 13, 2025	
FINAL REPORT	
August 20, 2025	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	1	0	1	0	0
Low	6	0	4	0	2
Informational	6	0	4	0	2
All severities	13	0	9	0	4

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Merkle Vester in `magna-eng/protocol`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Withdraw Allows Unauthorized Post-Claim Actions	Logical Error	MEDIUM	ACKNOWLEDGED	p. 8
L-01	RevokeAll Allows Draining New Vesting Schedules	Validation	LOW	ACKNOWLEDGED	p. 9
L-02	Potential Global allocation.id Collisions	Validation	LOW	ACKNOWLEDGED	p. 10
L-03	Misleading Revert Reason	Warning	LOW	RESOLVED	p. 14
L-04	High-level Transient Variables	Warning	LOW	ACKNOWLEDGED	p. 16
L-05	Direct Claim Handler Address	Warning	LOW	ACKNOWLEDGED	p. 18
L-06	Misleading Immutable	Warning	LOW	RESOLVED	p. 19
I-01	Unused Imports	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 21
I-02	Missing License Declaration	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 23
I-03	Code Typos	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 24
I-04	Outdated Documentation	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 28
I-05	Lack Of Sanity Checks	Validation	INFORMATIONAL	RESOLVED	p. 31
I-06	Unused Variable Computation	Best Practices	INFORMATIONAL	RESOLVED	p. 32

MEDIUM **M-01**

ACKNOWLEDGED

Withdraw Allows Unauthorized Post-Claim Actions

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/protocol-magnameklevester-team2/blob/6d1a8217cc717a99fd6cbc3a3d47b381831deb86/evm/src/distribution/src/MerkleVester.sol#L374
REVIEW	Main Review

DESCRIPTION

The `withdraw` function can be executed by any caller on behalf of another user, using the target user's `rootIndex`, `decodableArgs`, and `proof`. However, the `postClaimHandler` and `extraData` params are not fixed by the user and can be arbitrarily set by the caller.

If the `postClaimHandlerWhitelist` contains both direct claim options and contracts such as staking contracts, a caller could force a user's withdrawal to be routed into staking—potentially with a lock-up period. Similarly, the caller could choose `extraData` values that, depending on the handler contract's logic, could negatively affect the user's funds without their consent.

RECOMMENDATION

If the withdrawal is not a direct claim, restrict execution to the `withdrawalAddress`.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

RevokeAll Allows Draining New Vesting Schedules

CATEGORY Validation
LOCATION MerkleVester.sol
REVIEW Main Review

DESCRIPTION

The `revokeAll` function transfers the entire contract balance to the `benefactor` without updating allocation states or setting any revocation flags:

```
function revokeAll() external nonReentrant onlyRole(BENEFACTOR) {  
    SafeERC20.safeTransfer(IERC20(token), msg.sender, IERC20(token).balanceOf(address(this)));  
}
```

This leaves existing allocations' `DistributionState` unchanged (e.g., `terminatedTimestamp` remains 0), allowing beneficiaries of old allocations to continue computing vested amounts and withdrawing funds if the contract is later refunded for new allocations. Since funding is global (not per-allocation), old claims can drain the pool on a first-come-first-served basis, rendering new allocations insolvent despite valid vesting schedules. No events are emitted, reducing transparency and the function can be called repeatedly, exacerbating risks in underfunded or multi-root scenarios. This can lead to loss of funds for new beneficiaries.

RECOMMENDATION

Update `revokeAll` to set a global revocation flag (e.g., a bool `allRevoked` in storage) that blocks all future calls to `addAllocationRoot`, `setAllocationRoot` and `fund` functions. Alternatively, pair it with pausing the contract. Finally, consider emitting a `RevokedAll` event with the transferred amount for auditability.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Potential Global allocation.id Collisions

CATEGORY	Validation
LOCATION	MerkleVester.sol
REVIEW	Main Review

DESCRIPTION

All mutable vesting state is stored under a single key: the free form string `allocation.id`, shared across the entire contract lifetime irrespective of which Merkle root/batch a leaf belongs to. The contract never namespaces or fingerprints the state by rootIndex, root hash, or beneficiary. It also never enforces that a later leaf reusing the same id refers to the same logical allocation.

Relevant storage and lookups:

```
// MerkleVester.sol
mapping(string => DistributionState) public schedules; // mutable state (withdrawn,
termination, withdrawalAddress)
mapping(string => bool) private feeAlreadyPaid; // fee "pay once" flag
// Everywhere state is read/written, the key is only allocation.id:
schedules[allocation.id].withdrawalAddress
schedules[allocation.id].terminatedTimestamp
schedules[allocation.id].withdrawn
feeAlreadyPaid[allocation.id]
```

Operationally, it is very common for teams (or different admin wallets, see [the following link] (<https://docs.magna.so/admins/product-guides/magna-quickstart-guide/step-1-set-up-key-project-details/step-1c-how-to-add-admin-wallet-s>)) to build multiple Merkle trees over time (e.g., "Grants 2025", "Airdrop Wave 2") and to reuse human-friendly IDs like "Distribution#2025". Because the contract treats id as a global primary key, any two leaves in different roots that share the same id will share one `DistributionState` and one fee flag. The first claimant to touch that id determines the `withdrawalAddress` used for all future claims with that id, regardless of which root those future claims come from. Admin actions (cancel/revoke) and the "pay fee once" bit also bleed across batches.

Impact:

1. Funds can be misdirected: a claim under batch A sets `schedules[id].withdrawalAddress` and claims under batch B (different person) will pay to that same address.

L-02 Potential Global allocation.id Collisions

CONTINUED

2. Future vesting can be capped/disabled across batches: `cancel` / `revoke` of one leaf sets `terminatedTimestamp` for all leaves sharing that id.
3. Fees can be under-collected: claiming once anywhere sets `feeAlreadyPaid[id]=true` for all other leaves with that id.

This is a correctness flaw that can permanently redirect funds or block rightful beneficiaries and it is realistic in production (distinct teams/batches often reuse friendly IDs).

Proof of Concept

Assume two completely separate Merkle roots are deployed:

- Root #0 ("HR Salaries 2025", built by HR → admin wallet #1)

```
Allocation {
  id: "salaries#2025",
  originalBeneficiary: 0xA11c...ALEX,
  totalAllocation: ...,
  transferableByBeneficiary: true,
  ... // schedule = INTERVAL over 12 months
}
```

- Root #1 ("Growth Salaries 2025", built by Growth → admin wallet #2)

```
Allocation {
  id: "salaries#2025",
  originalBeneficiary: 0xB0bB...B0B,
  totalAllocation: ...,
  transferableByBeneficiary: true,
  ... // schedule = CALENDAR single unlock after 30 days
}
```

Both leaves are valid for their respective roots. The id collision is accidental (common when using human-friendly IDs).

T0 — Deploy roots

```
merkleRoots[0] = <root for HR Salaries 2025>
merkleRoots[1] = <root for Growth Salaries 2025>
```

On-chain state is empty for this id:

```
schedules["salaries#2025"] = { withdrawalAddress: address(0), terminatedTimestamp: 0,
withdrawn: 0, ... }
```

T1 — Bob claims his airdrop first (rootIndex = 1)

L-02 Potential Global allocation.id Collisions

CONTINUED

```

withdraw(
  0, // all available
  1, // Growth salary root
  abi.encode("calendar", bobAlloc, bobSchedule),
  proofForRoot1,
  DIRECT_CLAIM_HANDLER,
  ""
);

```

Inside withdraw:

- The Merkle proof for root #1 validates.
- `_checkOrSetOriginalBeneficiary(bobAlloc)` runs:

```

if (schedules[bobAlloc.id].withdrawalAddress == address(0)) {
  schedules[bobAlloc.id].withdrawalAddress = bobAlloc.originalBeneficiary; // 0xB0bB...B0B
}

```

- Tokens are transferred to `schedules["salaries#2025"].withdrawalAddress` (now Bob).

Result: Bob is paid and the global `withdrawalAddress` for "salaries#2025" is pinned to Bob.

T2 — Weeks later, Alice claims her salary (rootIndex = 0)

```

withdraw(
  0, // all available
  0, // HR salary root
  abi.encode("interval", aliceAlloc, aliceSchedule),
  proofForRoot0,
  DIRECT_CLAIM_HANDLER,
  ""
);

```

Inside withdraw:

1. The Merkle proof for root #0 validates.
2. `_checkOrSetOriginalBeneficiary(aliceAlloc)` does nothing because `schedules[id].withdrawalAddress != 0`.
3. Payout destination is looked up only from global state:

```

address to = schedules["salaries#2025"].withdrawalAddress; // 0xB0bB...B0B
SafeERC20.safeTransfer(token, to, aliceVestedNow);

```

L-02 Potential Global allocation.id Collisions

CONTINUED

Alice's salary is transferred to Bob's address. Alice never needed to lose a key or sign anything, the only cause is that both leaves reused "salaries#2025".

— RECOMMENDATION

Consider documenting this risk in the `addAllocationRoot` and `setAllocationRoot` functions. Implement safeguards at your backend that ensures that 2 ids are never reused.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

RESOLVED

Misleading Revert Reason

CATEGORY	Warning
LOCATION	MerkleVester.sol
REVIEW	Main Review

DESCRIPTION

In `_transferBeneficiaryAddress` the code intends to block beneficiary-initiated transfers while the contract is paused, allowing only admin-initiated transfers in that state. The guard is functionally correct but the revert reason is inverted: it reverts with `ContractNotPaused` when `paused() == true`.

Current code:

```
bool authorizedByAdmin = (AccessControl.hasRole(BENEFACTOR, msg.sender) &&
allocation.transferableByAdmin);
bool authorizedByBeneficiary = (msg.sender == state.withdrawalAddress &&
allocation.transferableByBeneficiary);
// authorizedByBeneficiary can only be true if not paused, error that should be used is
ContractPaused()
if (authorizedByBeneficiary && !authorizedByAdmin && paused()) revert ContractNotPaused();
```

Elsewhere in the codebase, `ContractNotPaused()` is used to enforce "this action requires the contract to be paused" (e.g., `setAllocationRoot`), which makes the above usage inconsistent and confusing.

RECOMMENDATION

Emit a revert that accurately reflects the state and keep semantics consistent across the codebase.

Introduce a `ContractPaused()` custom error and use it here, or reuse a consistent error/message used for "paused" across the project.

Add the error in `AirlockTypes.sol`:

```
error ContractPaused();
```

Use this new custom error in the `_transferBeneficiaryAddress` function:

```
function _transferBeneficiaryAddress(
DistributionState storage state,
```

L-03 Misleading Revert Reason

CONTINUED

```
Allocation memory allocation,  
address _newAddress  
) internal {  
...  
if (authorizedByBeneficiary && !authorizedByAdmin && paused()) revert ContractPaused();  
...  
}
```

— RESOLUTION

Status: Resolved.

LOW **L-04**

ACKNOWLEDGED

High-level Transient Variables

CATEGORY	Warning
LOCATION	MerkleVester.sol
REVIEW	Main Review

DESCRIPTION

The code uses high-level transient storage variables whose semantics compile down to EIP-1153 TSTORE/TLOAD and are only code-generated by Solidity starting with 0.8.28. The files declare a floating pragma `^0.8.24`, which allows any compiler in the range `>=0.8.24 <0.9.0`. If CI, verification, or a local build picks 0.8.24–0.8.27, high-level transient variables are not supported for code generation and builds will fail or behave unexpectedly. Even when compiled with 0.8.28+, deploying to a network that has not activated Cancun (EIP-1153) will cause runtime failures because the emitted opcodes are not available pre-fork.

Representative snippets:

```
// MerkleVester.sol
pragma solidity ^0.8.24;
bool transient insideMulticall;
bool transient transientFeeInitialized;
uint256 transient transientFeeReceived;
uint256 transient transientFeeIncurred;
```

These transient fields are central to fee accounting and the multicall lifecycle. Compiler drift here directly affects correctness: different minor compilers can accept or reject transient, generate different IR/Yul, or target an evmVersion that does not include Cancun.

RECOMMENDATION

Pin the compiler to a specific version that supports high-level transient storage and set the EVM to Cancun in your toolchain. Replace floating pragmas with an exact version and enforce the same in your build config. For example, update all contracts and configuration as follows:

```
// In every Solidity file
pragma solidity 0.8.28;

# foundry.toml
[profile.default]
```

L-04 High-level Transient Variables

CONTINUED

```
solc_version = "0.8.28"
evm_version = "cancun"
optimizer = true
optimizer_runs = 200

// hardhat.config.ts
import { HardhatUserConfig } from "hardhat/config";
const config: HardhatUserConfig = {
  solidity: {
    version: "0.8.28",
    settings: { evmVersion: "cancun", optimizer: { enabled: true, runs: 200 } }
  }
};
export default config;
```

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Direct Claim Handler Address

CATEGORY	Warning
LOCATION	MerkleVester.sol
REVIEW	Main Review

— DESCRIPTION

The `MerkleVester` contract supports direct claims (i.e., transferring tokens directly to the beneficiary without a post-claim handler) via the `DIRECT_CLAIM_HANDLER` constant set to `address(0)`. A convenience overload of the `withdraw` function uses this handler implicitly. However, the internal `_withdrawToBeneficiary` function requires all post-claim handlers, including `address(0)`, to be explicitly whitelisted in the `postClaimHandlerWhitelist`. The constructor only whitelists handlers provided in the `_postClaimHandlers` array and does not automatically add `address(0)`. If deployment scripts omit `address(0)` from this array, attempts to use the default `withdraw` overload (or explicitly pass `address(0)`) will revert with `PostClaimHandlerNotWhitelisted()`. This breaks the "plain" withdrawal functionality, which is intended as a user-friendly default.

Users expecting simple, direct withdrawals (without custom handlers) will encounter unexpected reverts, leading to failed transactions.

— RECOMMENDATION

Consider automatically whitelisting `address(0)` in the constructor to enable direct claims by default:

```
// In constructor, after adding provided handlers:  
postClaimHandlerWhitelist.add(address(0)); // Enable direct claims out-of-the-box
```

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

RESOLVED

Misleading Immutable

CATEGORY	Warning
LOCATION	MerkleVester.sol
REVIEW	Main Review

DESCRIPTION

In the `MerkleVester` contract, an immutable `feeSetter` address is set during construction and documented as the "address who can update the fee." If `_feeSetter` is non-zero, the constructor grants the `FEE_SETTER_ROLE` to this address. However, the `setClaimFee` function enforces access solely via `onlyRole(FEE_SETTER_ROLE)`, without any direct reference to the `feeSetter` address. This role-based access (inherited from OpenZeppelin's `AccessControl`) allows the contract's admin (holding `DEFAULT_ADMIN_ROLE`, initially the benefactor) to grant or revoke the role to/from any address, including multiple parties.

The `feeSetter` variable itself is never read or used after the constructor, rendering it dead code. This creates a discrepancy between the code's behavior (flexible, role-based fee setting) and its documentation/immutable storage (implying a fixed, address-specific privilege). In the outlined valid states for fees:

- Case 3 (`feeCollector != address(0), feeSetter != address(0)`): Fees are updatable, but not exclusively by `feeSetter`, any role holder can update. This could mislead deployers, auditors or users into assuming `feeSetter` has exclusive, immutable control, leading to incorrect trust assumptions or overlooked admin privileges in fee management.

The flow is as follows:

1. Constructor: Set `feeSetter = _feeSetter`; if non-zero, grant `FEE_SETTER_ROLE` to it.
2. Fee updates: Checked only against the role, not the address.
3. Role management: Admin can alter role holders at any time, bypassing the immutable `feeSetter`.

No security vulnerability exists if roles are managed correctly, but the unused variable and docs inconsistency reduce clarity.

RECOMMENDATION

Remove the immutable `feeSetter` variable entirely, as it serves no functional purpose post-construction. If retaining for logging/initial setup reference, rename it to `initialFeeSetter` and update all documentation (e.g., constructor params, valid states) to

L-06 Misleading Immutable

CONTINUED

explicitly state: "Fee updates are controlled by holders of `FEE_SETTER_ROLE`, initially granted to the provided address if non-zero. The admin (benefactor) can grant/revoke this role to manage access flexibly." Add an event emission in the constructor for the initial grant to improve traceability.

— RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Unused Imports

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Several Solidity files pull in libraries or contracts they never reference. Keeping unused imports makes the code harder to review and marginally increases compile time. Below are the concrete instances found in the code you provided, with the exact lines and why they are unused.

In `MerkleVester.sol`, the file imports OpenZeppelin's `MerkleProof` but never calls it here. Merkle verification is delegated to `MerkleValidator`. The import can be safely removed from this unit.

```
// MerkleVester.sol
import {MerkleProof} from "@openzeppelin/utils/cryptography/MerkleProof.sol"; // ← unused in this file
```

In `IAirlockBase.sol`, the `Math` library is included but not used anywhere in the contract. All arithmetic is plain checked arithmetic or handled by `SafeERC20` / `AccessControl` logic, there are no calls to `Math.*` in this unit.

```
// IAirlockBase.sol
import "@openzeppelin/utils/math/Math.sol"; // ← unused
```

In `IMerkleVester.sol`, the interface does not invoke or reference `SafeERC20` at all, so that import is unused.

```
// IMerkleVester.sol
import "@openzeppelin/token/ERC20/utils/SafeERC20.sol"; // ← unused
```

In `AirlockTypes.sol`, the file defines only errors and structs, yet it imports four libraries/contracts it never uses. None of `SafeERC20`, `AccessControl` or `ReentrancyGuard` are referenced in this file and can be removed entirely.

```
// interfaces/AirlockTypes.sol
import "@openzeppelin/token/ERC20/utils/SafeERC20.sol"; // ← unused
import {AccessControl} from "@openzeppelin/access/AccessControl.sol"; // ← unused
import {ReentrancyGuard} from "@openzeppelin/utils/ReentrancyGuard.sol"; // ← unused
```

I-01 Unused Imports

CONTINUED

— RECOMMENDATION

Consider removing the unused imports described above.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Missing License Declaration

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The codebase lacks any form of license declaration. This includes:

- No SPDX-License-Identifier comments at the top of Solidity files (e.g., `// SPDX-License-Identifier: MIT`).
- No LICENSE file in the root directory or subdirectories.
- No mentions of licensing terms in the `README.md` or other documentation files.

This absence creates ambiguity regarding the legal terms under which the code can be used, modified, distributed, or contributed to. Without a license, the code defaults to "all rights reserved" under copyright law, potentially restricting adoption and collaboration.

RECOMMENDATION

Choose and declare a license for the project, then make it explicit in every source file and at the repository root.

1. Add a `LICENSE` file at the repo root that matches your intended terms. Common options:

- MIT (most compatible with OpenZeppelin and widely used in web3)
- Apache-2.0 (permissive + patent grant)
- GPL-3.0-or-later (copyleft)
- BUSL-1.1 (source-available, time-delayed commercial use)

2. Add an SPDX header to the top of every `.sol` file, before the pragma. For example, if you choose MIT:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;
```

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Code Typos

CATEGORY	Documentation
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Multiple files contain spelling mistakes, grammar slips and a few poorly named symbols. While these don't change runtime behavior, they can confuse reviewers and integrators and in a few places could mislead automation that keys off error names or comments. Below is a concrete inventory with suggested corrections:

`MerkleVester.sol`

Typo in comments and parameter docs; also "paid" vs "payed":

```
/// @dev lazily store the mutable state as allocaitons are interacted with
// ^^^^^^^^^^^ → allocations
/**
 * @param benefactor initial administator and benefactor of the contract
 * ^^^^^ → initial ^^^^^^^^^^^^^ → administrator
 * @param _shouldPayClaimFeeOnlyOnce true, if claim fee should only be paid for the first
claim
 * ^^^ → paid
 */
/// @dev mapping allocation id to a boolean to store allocation ids for which fee was already
payed
// ^^^ → paid
// In setAllocationRoot() comment:
// "whith the new modified schedule" → "with the new modified schedule"
/**
 * @notice Returns true, if the decodableArgs is calendar type
 * @param decodableArgs decodable args to checks whether it is calendar type or not
 * ^^^^^ → check
 */
IAirlockBase.sol
```

I-03 Code Typos

CONTINUED

Constructor docs, variable name casing and a misspelled word in a comment.

```
/**
 * @param _benefactor initial administator and benefactor of the contract
 * ^^^^^^ → initial
 * ^^^^^^^^^^^^^ → administrator
 */
uint256 _postClaimHandlersLength = _postClaimHandlers.length;
// ^^ Wrong casing → _postClaimHandlersLength (non-functional, but tidy)
...
// any error in the postClam handler will revert the entire transaction
// ^^^^ → postClaim
```

Function naming is also awkward:

```
function removePostClaimHandlerToWhitelist(...) external { ... }
// "ToWhitelist" reads like "add". Prefer removePostClaimHandlerFromWhitelist(...) for
clarity
```

IMerkleVester.sol

Doc typos in returns and comments.

```
/**
 * @return the new lenght of the number of merkle roots array minus 1
 * ^^^^^^ → length
 */
/**
 * @dev using defund can result in underfunding the total liabilies of the allocations
 * ^^^^^^^^^ → liabilities
 */
```

interfaces/AirlockTypes.sol

Several repeated slips in comments and error docs.

```
/// @dev solidity does not support immutablability outside of compile time
// ^^^^^^^^^^^^^^^^^ → immutability (appears multiple times)
/// @dev error thrown when to many timestamps are provided
// ^^ → too
/// @dev error thrown when the supplied beneficiary address is the same as an others
// ^^^^^^ → another's
```

I-03 Code Typos

CONTINUED

```

/// @dev error thrown when an an amount in an interval is invalid
// ^^ → an
/// @dev error thrown when the the claim fee handler is not yet whitelisted
// ^^ → the
/// @param totalAllocation total amount of tokens to vest in the allocaiton
// ^^^^^^^^^ → allocation
/// @param fundedAmount ... merkle vester does not support funding indivual allocations
// ^^^^^^^^^ → individual
// same "indivual" typo repeats in terminatedWithdrawn/terminatedAmount comments

interfaces/ICalendarVester.sol

```

Header comment misspells "Calendar".

```

/// @notice Abstract contract to define common behavior for Calender type vesters
// ^^^^^^^ → Calendar

MerkleValidator.sol

```

Duplicated word in doc comment.

```

/// @notice Verify verify a leaf is included in the Merkle tree
// ^^^^^ → (remove duplicate) "Verify a leaf ..." or "Verifies that a leaf ..."

README.md

```

Multiple typos and a subject/verb agreement slip, plus a word choice that doesn't match code semantics.

```

Vester contracts internally operates on allocations
# ^^^^^ → operate
Merkle based contracts can provide gas savings when the number of allocatoins are high
# ^^^^^^^^^ → allocations
Interval schedules can be ... subdiveded into a piece
# ^^^^^^^^^ → subdivided
## 1.4 Noteable features
# ^^^^^ → Notable
... indivual ...
# ^^^^^ → individual

```

I-03 Code Typos

CONTINUED

The `README` also uses “percents”/percentages in examples for calendar/interval, while the code uses absolute unlockAmounts and piece amount. That’s not a spelling error but a terminology mismatch worth fixing to avoid builder confusion.

— RECOMMENDATION

Standardize and clean up language typos across the repository.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Outdated Documentation

CATEGORY Documentation
LOCATION Global
REVIEW Main Review

DESCRIPTION

Docs show calendar/interval examples in percents, but the code uses absolute amounts:

```
// evm/src/distribution/README.md
## 1.2 Types of vesting schedules
There are two types of schedules implemented.
### 1.2.1 Calendar schedules
/**
 * @notice Immutable unlock schedule for calendar allocations
 * @dev solidity does not support immutability outside of compile time, contracts must not
implement mutability
 *
 * @param unlockScheduleId id of the allocation
 * @param unlockTimestamps sequence of timestamps when funds will unlock
 * @param unlockPercents sequence of percents that unlock at each timestamp, in 10,000ths
 */
struct CalendarUnlockSchedule {
    string unlockScheduleId;
    uint32[] unlockTimestamps;
    uint256[] unlockPercents;
}
```

vs actual code:

```
// AirlockTypes.sol
/**
 * @notice Immutable unlock schedule for calendar allocations
 * @dev solidity does not support immutability outside of compile time, contracts must not
implement mutability
 *
 * @param unlockScheduleId id of the allocation
 * @param unlockTimestamps sequence of timestamps when funds will unlock
```

I-04 Outdated Documentation

CONTINUED

```
* @param unlockAmounts sequence of amounts that unlock at each timestamp
*/
struct CalendarUnlockSchedule {
string unlockScheduleId; // Workaround for Internal or recursive type is not allowed for
public state variables
uint32[] unlockTimestamps;
uint256[] unlockAmounts;
}
```

Moreover interval docs show percent per piece, code has amount:

```
// evm/src/distribution/README.md
### 1.2.2 Interval schedules
/**
* @notice Immutable unlock schedule for interval allocations
* @dev solidity does not support immutability outside of compile time, contracts must not
implement mutability
*
* @param unlockScheduleId id of the allocation
* @param pieces sequence of pieces representing phases of the unlock schedule, percents of
pieces must sum to 100%
*/
struct IntervalUnlockSchedule {
string unlockScheduleId; // Workaround for Internal or recursive type is not allowed for
public state variables
Piece[] pieces;
}
/**
* @notice Represents a phase of an interval unlock schedule
* @dev solidity does not support immutability outside of compile time, contracts must not
implement mutability
*
* @param startDate start timestamp of the piece
* @param periodLength time length of the piece
* @param numberOfPeriods how many periods for this piece
* @param percent the total percent, in 10,000ths that will unlock over the piece
*/
struct Piece {
uint32 startDate;
uint32 periodLength;
```

I-04 Outdated Documentation

CONTINUED

```
uint32 numberOfPeriods;  
uint32 percent;  
}
```

vs actual code:

```
// AirlockTypes.sol  
/**  
 * @notice Represents a phase of an interval unlock schedule  
 * @dev solidity does not support immutability outside of compile time, contracts must not  
implement mutability  
 *  
 * @param startDate start timestamp of the piece  
 * @param periodLength the length of each period in seconds  
 * @param numberOfPeriods the number of periods in the piece  
 * @param amount the amount of tokens that is released in each piece  
 */  
struct Piece {  
uint32 startDate;  
uint32 periodLength;  
uint32 numberOfPeriods;  
uint256 amount;  
}
```

— RECOMMENDATION

Consider updating examples and field names in docs to avoid confusion.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

RESOLVED

Lack Of Sanity Checks

CATEGORY	Validation
LOCATION	ICalendarVester.sol
REVIEW	Main Review

DESCRIPTION

The `_getVestedAmount` function in the `CalendarVester` abstract contract does not enforce sanity checks on the input arrays for calendar schedules. Specifically, there are no validations to ensure that `unlockTimestamps.length` equals `unlockAmounts.length`, or that the `unlockTimestamps` array is strictly increasing (sorted in ascending order). Since schedule data originates from off-chain Merkle leaves (validated only by proof inclusion), malformed inputs could lead to incorrect vesting calculations, such as under-vesting, over-vesting, or transaction reverts due to out-of-bounds array access. While the benefactor controls Merkle roots and is trusted to provide valid data, the absence of on-chain checks increases the risk of errors from backend misconfigurations or adversarial leaves.

RECOMMENDATION

- Add explicit validations in `_getVestedAmount` (or at the point of decoding in `getCalendarLeafAllocationData`):

```
require(_unlockTimestamps.length == _unlockAmounts.length, "Array length mismatch");
for (uint256 i = 1; i < _unlockTimestamps.length; i++) {
    require(_unlockTimestamps[i-1] < _unlockTimestamps[i], "Timestamps not strictly increasing");
}
```

- Since data is from trusted Merkle roots, these checks could be optional or emit events on failure for monitoring. Alternatively, enforce sanity off-chain during Merkle tree construction, but document the assumption clearly.
- Update backend tools/scripts to validate schedules before root generation.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-06

RESOLVED

Unused Variable Computation

CATEGORY	Best Practices
LOCATION	IIntervalVester.sol
REVIEW	Main Review

DESCRIPTION

In the `_getVestedAmount` function of the `IntervalVester` abstract contract, the line `uint256 vestingEndTimestamp = _getPieceEndTime(schedule.pieces[schedule.pieces.length - 1]);` calculates the end timestamp of the last piece in the unlock schedule by invoking `_getPieceEndTime` on the final element of the `pieces` array. This computation converts the result (a `uint32`) to a `uint256` and assigns it to a local variable `vestingEndTimestamp`. However, this variable is never referenced or utilized anywhere else in the function body, the return statement, or subsequent logic.

This constitutes dead code, which may be a remnant from an earlier implementation where the end timestamp was used (e.g., to cap the `finalTimestamp` or perform additional checks). While harmless in terms of correctness (the function still accurately sums vested amounts across pieces), it incurs unnecessary computational overhead and increased gas costs.

RECOMMENDATION

Remove the line entirely to eliminate dead code and associated overhead. If the computation was intended for a purpose (e.g., validating the schedule's overall end time or capping vesting), reintroduce it with explicit usage, such as `require(finalTimestamp <= vestingEndTimestamp, "InvalidTimestamp");` or integrating it into the return cap.

RESOLUTION

Status: Resolved.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.